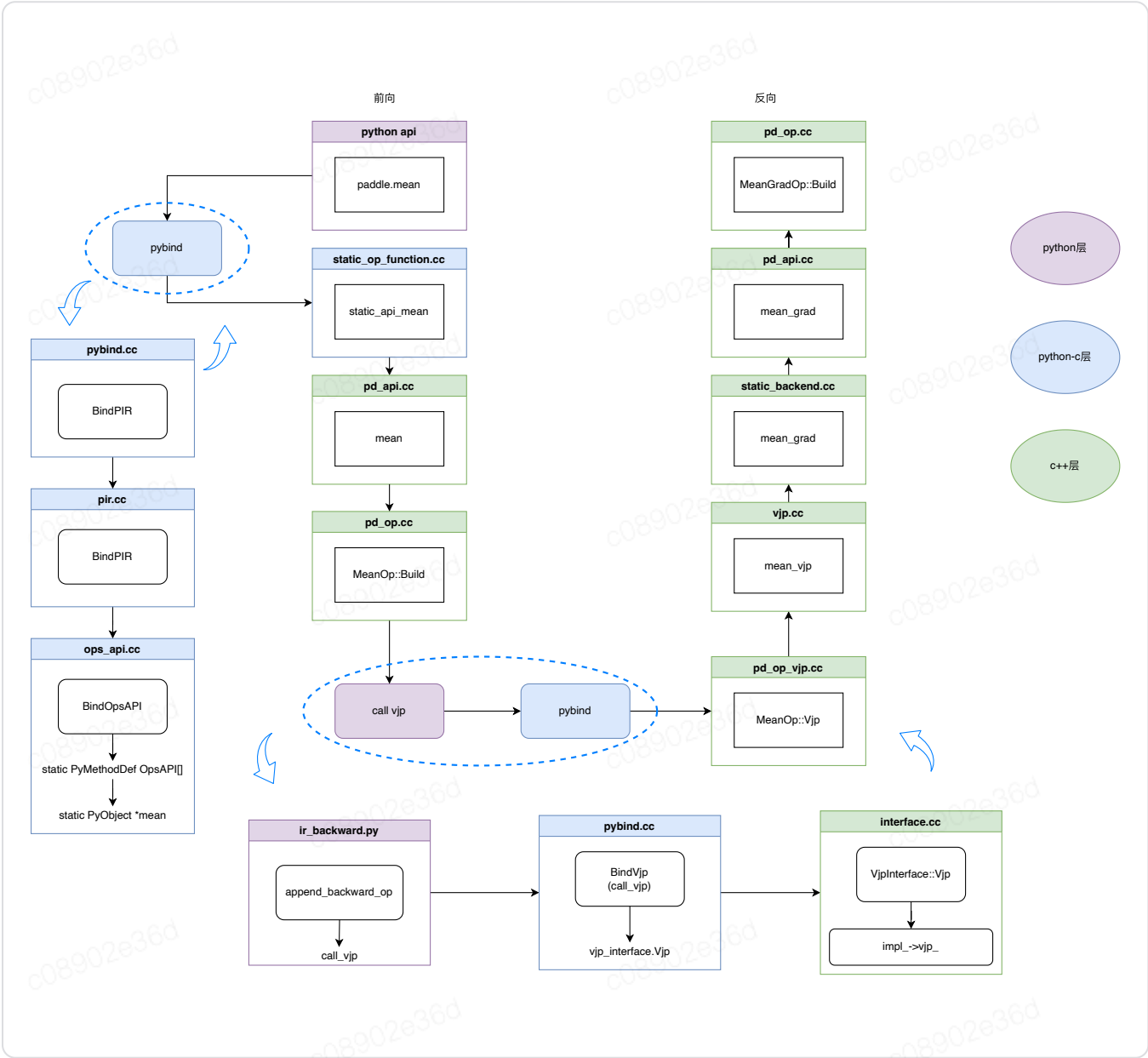


新IR下前反向组网笔记梳理

整体架构



前向组网:

参考[NewIR Python端设计方案](#)

前向组网的核心是如何通过Python调用来将一个算子插入Program拓扑结构中，在旧IR体系中，我们是通过在Python端调用统一接口`append_op`，通过传入所需要的信息，去构建一个`OpDesc`

再插入到Program中；

在新IR体系中，为了能够符合新IR设计理念，提高框架组网代码可读性与一致性，同时减少不兼容问题产生，我们使用类似函数式接口的设计方式来进行组网，同时为了简化使用，保持动静接口统一，降低维护成本，我们将动态图和新IR静态图统一到_C_ops进行调用。

e.g.

用户调用python_api中的mean算子：

</>

Python | 收起 ^

```
1 >>> import paddle
2
3 >>> x = paddle.to_tensor([[[[1., 2., 3., 4.],
4 ...                        [5., 6., 7., 8.],
5 ...                        [9., 10., 11., 12.]],
6 ...                       [[13., 14., 15., 16.],
7 ...                       [17., 18., 19., 20.],
8 ...                       [21., 22., 23., 24.]]]])
9 >>> out1 = paddle.mean(x)
10 >>> print(out1.numpy())
11 12.5
```

在新IR下Python端插入Op，paddle.mean里调用op

 python/paddle/tensor/

</> stat.py

Python | 收起 ^

```
1 def mean(x, axis=None, keepdim=False, name=None):
2     # 其他非关键代码省略
3     if in_dynamic_or_pir_mode():
4         return _C_ops.mean(x, axis, keepdim) # 动静接口统一
```

Pybind层，C++ bind到python端

 paddle/fluid/pybind/

</> pybind.cc

C++ | 收起 ^

```
1 PYBIND11_MODULE(libpaddle, m) {  
2     .....  
3     BindPIR(&m);           //pybind  
4     BindVjp(&m);  
5 }
```

</> pir.cc

C++ | 收起 ^

```
1 void BindPIR(pybind11::module *module) {  
2     auto ir_module = module->def_submodule("ir");  
3     BindProgram(&ir_module);  
4     BindBlock(&ir_module);  
5     BindOperation(&ir_module);  
6     BindValue(&ir_module);  
7     BindOpOperand(&ir_module);  
8     BindOpResult(&ir_module);  
9     BindType(&ir_module);  
10    BindUtils(&ir_module);  
11    BindIrPass(&ir_module);  
12    BindPassManager(&ir_module);  
13    auto ops_modules = ir_module.def_submodule("ops");  
14    BindOpsAPI(&ops_modules);  
    //bind API  
15 }
```

```
1 void BindOpsAPI(pybind11::module *module) {
2     if (PyModule_AddFunctions(module->ptr(), OpsAPI) < 0) {
3         PADDLE_THROW(phi::errors::Fatal("Add C++ api to core.ops failed!"));
4     }
5     if (PyModule_AddFunctions(module->ptr(), ManualOpsAPI) < 0) {
6         PADDLE_THROW(phi::errors::Fatal("Add C++ api to core.ops failed!"));
7     }
8 }
9
10 static PyMethodDef OpsAPI[] = {
11     .....,
12     {"mean",
13      (PyCFunction)(void (*)(void))mean,
14      METH_VARARGS | METH_KEYWORDS,
15      "C++ interface function for mean."},
16 }
17
18 static PyObject *mean(PyObject *self, PyObject *args, PyObject *kwargs)
19 {
20     if (egr::Controller::Instance().GetCurrentTracer() == nullptr) {
21         //动静分离
22         VLOG(6) << "Call static_api_mean";
23         return static_api_mean(self, args, kwargs);
24     } else {
25         VLOG(6) << "Call eager_api_mean";
26         return eager_api_mean(self, args, kwargs);
27     }
28 }
```

</> static_op_function.cc

C++ | 收起 ^

```
1 PyObject *static_api_mean(PyObject *self, PyObject *args, PyObject
2 *kwargs) {
3     try {
4         VLOG(6) << "Add mean op into program";           //将mean算
5         //子插入到program中
6         VLOG(8) << "args count: " << (PyTuple_Size(args) / 2);
7
8         // Get OpResult from args
```

```

7      PyObject *x_obj = PyTuple_GET_ITEM(args, 0);
    //从PyObject类型的参数中转换出mean op需要的数据类型
8      auto x = CastPyArg2OpResult(x_obj, "mean", 0);
9
10     // Parse Attributes
11     PyObject *axis_obj = PyTuple_GET_ITEM(args, 1);
12     std::vector<int64_t> axis = CastPyArg2Longs(axis_obj, "mean",
13     1);
14     PyObject *keepdim_obj = PyTuple_GET_ITEM(args, 2);
15     bool keepdim = CastPyArg2Boolean(keepdim_obj, "mean", 2);
16
17     // Call ir static api
    //调用paddle api中的mean
18     auto static_api_out = paddle::dialect::mean(x, axis, keepdim);
19
20     return ToPyObject(static_api_out);
    //将结果转换为PyObject返回
21 } catch (...) {
22     ThrowExceptionToPython(std::current_exception());
23     return nullptr;
24 }

```

 paddle/fluid/ir/dialect/

</> pd_api.cc

C++ | 收起 ^

```

1 ir::OpResult mean(ir::OpResult x, const std::vector<int64_t>& axis, bool
  keepdim){
2     paddle::dialect::MeanOp mean_op =
  APIBuilder::Instance().GetBuilder()->Build<paddle::dialect::MeanOp>(x,
  axis, keepdim);
3     return mean_op.result(0);
4 }
    //调用paddle的meanOp, Build函数

```

pd_api.cc作用：作为Op build上一层，将build封装一下。同时为组合算子的使用提供便利。

</> pd_op.cc

C++ | 收起 ^

```

1 void MeanOp::Build(ir::Builder &builder, ir::OperationArgument
  &argument, ir::OpResult x_, ir::AttributeMap attributes) {
2
3     std::vector<int64_t> axis =
  attributes.at("axis").dyn_cast<paddle::dialect::IntArrayAttribute>
  ().data().GetData();

```

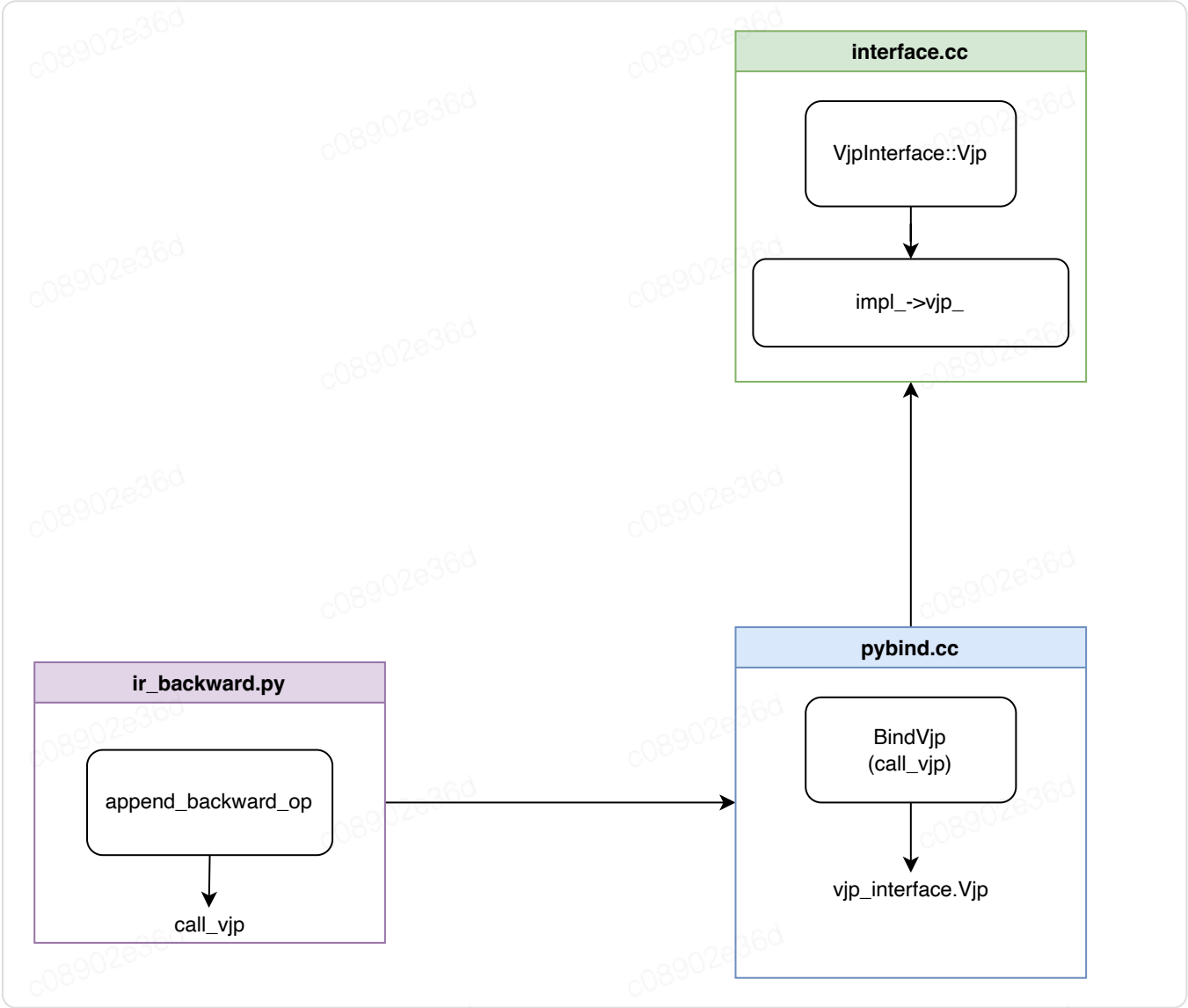
```
4
5   bool keepdim = attributes.at("keepdim").dyn_cast<ir::BoolAttribute>
   ().data();                                // attribute 类型转换
6
7
8   VLOG(4) << "Builder construction inputs";
9   std::vector<ir::OpResult> argument_inputs = {x_};
   //构建input
10  argument.AddOperands(argument_inputs.begin(), argument_inputs.end());
11
12  VLOG(4) << "Builder construction attributes";
13  ir::Attribute attr_axis =
   paddle::dialect::IntArrayAttribute::get(ir::IrContext::Instance(),
   phi::IntArray(axis));
14  argument.AddAttribute("axis", attr_axis);
   //构建attribute
15  ir::Attribute attr_keepdim =
   ir::BoolAttribute::get(ir::IrContext::Instance(), keepdim);
16  argument.AddAttribute("keepdim", attr_keepdim);
17
18  VLOG(4) << "Builder construction outputs";
   //构建output
19  paddle::dialect::DenseTensorType x =
   x_.type().dyn_cast<paddle::dialect::DenseTensorType>(); (void)x;
20
21  VLOG(4) << "Builder construction dense_x";
22  phi::DenseTensor
   dense_x(std::make_unique<paddle::experimental::DefaultAllocator>
   (paddle::platform::CPUPlace()).get(),
23
   phi::DenseTensorMeta(TransToPhiDataType(x.dtype()),
24
   x.dims(),
25
   x.data_layout(),
26
   x.lod(),
27
   x.offset()));
28  VLOG(4) << "Builder construction meta_x";                                //
   build 接收输入、属性, 借助于phi infermeta方法, 分析输出的类型和shape
29  phi::MetaTensor meta_x(&dense_x);
30  phi::DenseTensor dense_out;
31  phi::MetaTensor meta_out(&dense_out);
32
33  phi::ReduceIntArrayAxisInferMeta(meta_x, axis, keepdim, &meta_out);
34
35  std::vector<ir::Type> argument_outputs;
```

```
36  ir::Type out_dense_tensor_type =
    paddle::dialect::DenseTensorType::get(ir::IrContext::Instance(),
    TransToIrDataType(dense_out.dtype()), dense_out.dims(),
    dense_out.layout(), dense_out.lod(), dense_out.offset());
37  argument_outputs.push_back(out_dense_tensor_type);
38  argument.AddOutputs(argument_outputs.begin(), argument_outputs.end());
39
40 }
```

反向组网：

反向组网的核心是根据前向op列表，添加反向op列表。

VJP调用：call_vjp



 python/paddle/autograd/

</> ir_backward.py Python | 收起 ^

```
1 def append_backward_ops(
```

```
2     block, effective_forward_ops, no_grad_set, backward_ops, state
3 ):
4     ...
5     def update_input_grad_map(op, input_grads):
6         for i, input in enumerate(op.operands_source()):
7             if input.get_defining_op().name() == "builtin.combine":
8                 update_input_grad_map(input.get_defining_op(),
input_grads[i])
9             else:
10                input_grad = input_grads[i]
11                if isinstance(input_grad, list):
12                    state.value_to_valuegrad[input].append(input_grad)
13                else:
14                    state.value_to_valuegrad[input].append([input_grad])
15
16        # there are four patterns:
17        # [builtin.combine, op1] (op1's one input is vectorType, outputs
are not vectorType)
18        # [op2, builtin.split] (op2's inputs are not vectorType, one output
is vectorType)
19        # [builtin.combine, op3, builtin.split] (op3's one input and one
output are vectorType)
20        # [op4] (op4's inputs and outputs are not vectorType)
21        # einsum has two vectorType outputs, special pattern
22
23        clear_effective_forward_ops = []
24
25        for op in effective_forward_ops:
26            if op.name() != "builtin.combine" and op.name() !=
"builtin.split":
27                clear_effective_forward_ops.append(op)
28
29        for op in clear_effective_forward_ops:
30            if paddle.framework.core.has_vjp(op):
31                # prepare output_grad
32                output_grads = [] # (opresult)
33                zero_flag, output_grad = make_output_grad(op)
34                output_grads.append(output_grad)
35
36                # all(zero_flag) support this op has no contribution for
grad
37
38                # should be delete (prune sub_graph)
39                if len(output_grads) == 0 or all(zero_flag):
40                    continue
```



```

40
41     # prepare input_grad stop_gradient info.
42     input_grad_stopgradients = make_input_stopgradient(op)
43
44     # create grad_op
45     before_ops_num = len(block.ops)
46     input_grads = paddle.framework.core.call_vjp(
47         # 调用call_vjp
48         op, output_grads, input_grad_stopgradients
49     )
50     after_ops_num = len(block.ops)
51
52     # update grad_op structure
53     for i in range(before_ops_num, after_ops_num):
54         update_bwdop_structure(
55             backward_ops, state.op_to_opgrad[op], block.ops[i]
56         )
57
58     # update input_grad map
59     update_input_grad_map(op, input_grads)
60
61 else:
62     if op.num_operands() == 0 and op.num_results() != 0:
63         for value in op.results():
64             if len(state.value_to_valuegrad[value]) > 1:
65                 # need add sum op
66                 paddle.add_n(
67                     [
68                         item[0]
69                     ]
70                 )
71                 combineop = block.ops[len(block.ops) - 2]
72                 sumop = block.ops[len(block.ops) - 1]
73                 update_bwdop_structure(
74                     backward_ops, state.op_to_opgrad[op],
75                     combineop
76                 )
77                 update_bwdop_structure(
78                     backward_ops, state.op_to_opgrad[op], sumop
79                 )
80                 state.value_to_valuegrad[value] =
81                 [[sumop.result(0)]]

```

```

80         state.value_to_sumvaluegrad[
81             value
82         ] = state.value_to_valuegrad[value]
83     else:
84         state.op_to_opgrad[op] = []
85     else:
86         state.op_to_opgrad[op] = []
87

```

 paddle/fluid/pybind/

</> pybind.cc

C++ | 收起 ^

```

1 void BindVjp(pybind11::module *m) {
2     m->def(
3         "call_vjp",
4         [](pir::Operation &fwd_op,
5             //前向op信息
6             const std::vector<std::vector<pir::OpResult>> &out_grads,
7             const std::vector<std::vector<bool>> &stop_gradients) {
8             py::list res;
9             paddle::dialect::VjpInterface vjp_interface =
10                 fwd_op.dyn_cast<paddle::dialect::VjpInterface>();
11             //前向op信息中获取vjp interface
12             PADDLE_ENFORCE(
13                 vjp_interface,
14                 phi::errors::InvalidArgument(
15                     "The vjp function is not registered in %s op ",
16                     fwd_op.name())));
17             std::vector<std::vector<pir::OpResult>> vjp_res =
18                 vjp_interface.Vjp(&fwd_op, out_grads, stop_gradients);
19             //调用vjp_获取vjp结果
20             PADDLE_ENFORCE_EQ(
21                 stop_gradients.size(),
22                 vjp_res.size(),
23                 phi::errors::InvalidArgument(
24                     "The size of stop_gradients should be the same as
25                     vjp_res "
26                     "size."
27                     "But the size of stop_gradients: %d, vjp_res size: %d",
28                     stop_gradients.size(),
29                     vjp_res.size()));
30             for (size_t i = 0; i < vjp_res.size(); ++i) {

```

```

26         PADDLE_ENFORCE_EQ(stop_gradients[i].size(),
27                             vjp_res[i].size(),
28                             phi::errors::InvalidArgument(
29                                 "The size of stop_gradients[%d] should
        be the "
30                                 "same as vjp_res[%d] "
31                                 "size."
32                                 "But the size of stop_gradients[%d]: %d,
        "
33                                 "vjp_res[%d] size: %d",
34                                 i,
35                                 i,
36                                 i,
37                                 stop_gradients[i].size(),
38                                 i,
39                                 vjp_res[i].size()));
40         py::list sub_res;
41         for (size_t j = 0; j < vjp_res[i].size(); ++j) {
42             if (!vjp_res[i][j]) {
43                 sub_res.append(nullptr);
44             } else {
45                 sub_res.append(vjp_res[i][j]);
46             }
47         }
48         res.append(sub_res);
49     }
50     return res;
51 });
52
53 m->def("has_vjp", [](pir::Operation &fwd_op) {
54     pir::IrContext *ctx = pir::IrContext::Instance();
55     pir::OpInfo fwd_op_info = ctx->GetRegisteredOpInfo(fwd_op.name());
56     auto vjp_interface_impl =
57         fwd_op_info.GetInterfaceImpl<paddle::dialect::VjpInterface>();
58     if (vjp_interface_impl == nullptr) return false;
59     return true;
60 });
61

```

paddle/fluid/ir/dialect/paddle_dialect/interface/

</> vjp.h

C++ | 收起 ^

```
1 std::vector<std::vector<pir::OpResult>> Vjp(
```

```
2     pir::Operation* op,  
3     const std::vector<std::vector<pir::OpResult>>& out_grads,  
4     const std::vector<std::vector<bool>>& stop_gradients);  
    // 声明
```

</> interface.cc

C++ | 收起 ^

```
1 std::vector<std::vector<pir::OpResult>> VjpInterface::Vjp(  
    //实现  
2     pir::Operation* op,  
3     const std::vector<std::vector<pir::OpResult>>& out_grads,  
4     const std::vector<std::vector<bool>>& stop_gradients) {  
5     std::vector<std::vector<pir::Value>> out_grads_value;  
6     for (const auto& grad : out_grads) {  
7         std::vector<pir::Value> grad_value;  
8         for (auto op_result : grad) {  
9             grad_value.emplace_back(op_result);  
            //OpResult 转成 OpValue  
10        }  
11        out_grads_value.emplace_back(std::move(grad_value));  
12    }  
13    return impl_->vjp_(op, out_grads_value, stop_gradients);  
    //调用_vjp  
14 }  
15
```

</> pd_op.cc

C++ | 收起 ^

```
1 std::vector<std::vector<ir::OpResult>> MeanOp::Vjp(ir::Operation* op,  
    const std::vector<std::vector<ir::OpResult>>& out_grads, const  
    std::vector<std::vector<bool>>& stop_gradients){  
2     MeanOp op_obj = op->dyn_cast<MeanOp>();  
3  
4     VL0G(6) << "Prepare inputs of mean_grad";  
    //准备vjp输入（前向输入、输出、外部梯度）  
5  
    //输入类型  
6     Tensor x(std::make_shared<primitive::experimental::StaticTensor>  
        (op_obj.x()));  
7  
8     Tensor  
    out_grad(std::make_shared<primitive::experimental::StaticTensor>  
        (out_grads[0][0]));
```

```

9
10 VLOG(6) << "Vjp prepare Prepare attributes of mean_grad";
    //准备attribute
11
12     phi::IntArray axis = op-
    >attribute("axis").dyn_cast<paddle::dialect::IntArrayAttribute>
    ().data();
13     bool keepdim = op->attribute("keepdim").dyn_cast<ir::BoolAttribute>
    ().data();
14     bool reduce_all = false;
15
16 VLOG(4) << "Vjp prepare call mean's vjp inteface";
    //调用对应的xx_vjp接口
17     std::vector<std::vector<Tensor>> tensor_res =
18         primitive::experimental::mean_vjp(
19             x, out_grad, axis, keepdim, reduce_all, stop_gradients);
20
21 VLOG(4) << "Vjp prepare stop gradient of mean_grad";
    //构建返回结果, 与tensor_res维度一致, 已定义的结果直接赋值给res
22
23     std::vector<std::vector<ir::OpResult>> res(tensor_res.size());
24     for (size_t i = 0; i < tensor_res.size(); ++i) {{
25         res[i].resize(tensor_res[i].size());
26         for (size_t j = 0; j < tensor_res[i].size(); ++j) {{
27             if(tensor_res[i][j].defined()){{
28                 res[i][j] =
                std::static_pointer_cast<primitive::experimental::StaticTensor>
                (tensor_res[i][j].impl())->getValue().dyn_cast<ir::OpResult>();
29             }}
30         }}
31     }}
32     return res;
33 }

```

 paddle/fluid/primitive/rule/vjp/

</> (generated_)vjp.cc

C++ | 收起 ^

```

1 std::vector<std::vector<paddle::Tensor>> mean_vjp(const Tensor& x, const
  Tensor& out_grad, const IntArray& axis, bool keepdim, bool reduce_all,
  const std::vector<std::vector<bool>>& stop_gradients) {
2     std::vector<std::vector<paddle::Tensor>> vjp_res;
3     for (auto arg: stop_gradients) {
4         vjp_res.push_back(std::vector<paddle::Tensor>(arg.size()));
5     }

```

```

6  auto op_res = backend::mean_grad<LazyTensor>(x, out_grad, axis,
keepdim, reduce_all);    //vjp调用反向mean_grad得到op结果
7  vjp_res[0][0] = !stop_gradients[0][0] ? op_res : vjp_res[0][0];
//stop_gradient不需要赋值
8
9  return vjp_res;
10

```

 paddle/fluid/primitive/backend/

</> (generated_)static_backend.cc

C++ | 收起 ^

```

1  Tensor mean_grad<LazyTensor>(const Tensor& x, const Tensor& out_grad,
const IntArray& axis, bool keepdim, bool reduce_all) {
2  ir::OpResult x_res = std::static_pointer_cast<LazyTensor>(x.impl())-
>getValue().dyn_cast<ir::OpResult>();          //前向输出、外部
梯度进行类型转换
3  ir::OpResult out_grad_res = std::static_pointer_cast<LazyTensor>
(out_grad.impl())->getValue().dyn_cast<ir::OpResult>();
4  auto op_res = paddle::dialect::mean_grad(x_res, out_grad_res,
axis.GetData(), keepdim, reduce_all);
5  return Tensor(std::make_shared<LazyTensor>(op_res));
6
//调用mean_grad api
7 }

```

 paddle/fluid/ir/dialect/

</> pd_api.cc

C++ | 收起 ^

```

1  ir::OpResult mean_grad(ir::OpResult x, ir::OpResult out_grad, const
std::vector<int64_t>& axis, bool keepdim, bool reduce_all){
2  paddle::dialect::MeanGradOp mean_grad_op =
APIBuilder::Instance().GetBuilder()->Build<paddle::dialect::MeanGradOp>
(x, out_grad, axis, keepdim, reduce_all);
3  return mean_grad_op.result(0);
//api调用mean_grad_op
4 }

```

</> pd_op.cc

C++ | 收起 ^

```

1  void MeanGradOp::Build(ir::Builder &builder, ir::OperationArgument
&argument, ir::OpResult x_, ir::OpResult out_grad_, ir::AttributeMap
attributes) {
2

```

```
3  std::vector<int64_t> axis =
  attributes.at("axis").dyn_cast<paddle::dialect::IntArrayAttribute>
    ().data().GetData();
4
5  bool keepdim = attributes.at("keepdim").dyn_cast<ir::BoolAttribute>
    ().data();
6
7  bool reduce_all =
  attributes.at("reduce_all").dyn_cast<ir::BoolAttribute>().data();
8
9
10 VLOG(4) << "Builder construction inputs";
    //构建inputs
11  std::vector<ir::OpResult> argument_inputs = {x_, out_grad_};
12  argument.AddOperands(argument_inputs.begin(), argument_inputs.end());
13
14  VLOG(4) << "Builder construction attributes";
    //构建attributes
15  ir::Attribute attr_axis =
    paddle::dialect::IntArrayAttribute::get(ir::IrContext::Instance(),
    phi::IntArray(axis));
16  argument.AddAttribute("axis", attr_axis);
17  ir::Attribute attr_keepdim =
    ir::BoolAttribute::get(ir::IrContext::Instance(), keepdim);
18  argument.AddAttribute("keepdim", attr_keepdim);
19  ir::Attribute attr_reduce_all =
    ir::BoolAttribute::get(ir::IrContext::Instance(), reduce_all);
20  argument.AddAttribute("reduce_all", attr_reduce_all);
21
22  VLOG(4) << "Builder construction outputs";
    //构建outputs
23  paddle::dialect::DenseTensorType x =
    x_.type().dyn_cast<paddle::dialect::DenseTensorType>(); (void)x;
24  paddle::dialect::DenseTensorType out_grad =
    out_grad_.type().dyn_cast<paddle::dialect::DenseTensorType>();
    (void)out_grad;
25
26  VLOG(4) << "Builder construction dense_x";
27  phi::DenseTensor
    dense_x(std::make_unique<paddle::experimental::DefaultAllocator>
    (paddle::platform::CPUPlace()).get(),
28
    phi::DenseTensorMeta(TransToPhiDataType(x.dtype()),
29                                                                    x.dims(),
30                                                                    x.data_layout(),
```

```
31                                     x.lod(),
32                                     x.offset()));
33  VLOG(4) << "Builder construction meta_x";
34  phi::MetaTensor meta_x(&dense_x);
35  phi::DenseTensor dense_x_grad;
36  phi::MetaTensor meta_x_grad(&dense_x_grad);
37
38  phi::UnchangedInferMeta(meta_x, &meta_x_grad);
39
40  std::vector<ir::Type> argument_outputs;
41  ir::Type x_grad_dense_tensor_type =
42  paddle::dialect::DenseTensorType::get(ir::IrContext::Instance(),
43  TransToIrDataType(dense_x_grad.dtype()), dense_x_grad.dims(),
44  dense_x_grad.layout(), dense_x_grad.lod(), dense_x_grad.offset());
45  argument_outputs.push_back(x_grad_dense_tensor_type);
46  argument.AddOutputs(argument_outputs.begin(), argument_outputs.end());
47 }
```